
703DEMOV1: A Compound AI Agent for Cart-Abandonment Risk Recovery in Simulated E-Commerce

[TODO: author name, student ID, and group information]

COMPSCI 703

University of Auckland

[TODO: email or submission account]

Abstract

Cart abandonment is a commercially important e-commerce problem because a shopper who places products in a cart has already expressed purchase intent but may still leave before checkout. This report presents 703DEMOV1, a runnable Compound AI System that observes behavioural events in a simulated shopping website, maintains cart-scoped state, estimates abandonment risk, selects safe interventions, executes non-blocking website actions, and records feedback for evaluation. The implementation is not a chatbot: users do not send prompts to the system. Instead, the agent receives external behavioural input, updates a world model and belief state, plans candidate actions, applies safety constraints, and acts through a controlled frontend runtime. Gemini is used only as a bounded helper for low-score risk multiplier calibration; it is not the agent brain. The report describes the technical architecture, commercial logic, safety controls, token-cost strategy, evaluation plan, and reproducibility requirements for the final demo.

1 Introduction

Online shopping carts are not only purchase containers; prior consumer-behaviour research shows that users may also use carts for search, evaluation, entertainment, or organization before making a final purchase decision Kukar-Kinney and Close (2010). This makes cart abandonment both commercially valuable and technically subtle. A high-intent shopper may hesitate because of shipping cost, product uncertainty, seller trust, price sensitivity, checkout friction, or comparison with alternatives. A useful agent must therefore identify both the existence of abandonment risk and the likely cause.

703DEMOV1 addresses this problem through a simulated e-commerce website connected to an autonomous cart-abandonment recovery agent. The system is designed as a Compound AI System rather than a simple prompt-response interface. The Berkeley AI Research discussion of compound AI systems defines this class of systems as AI applications with multiple interacting components, including models, retrievers, and external tools Zaharia et al. (2024). Enterprise-oriented work similarly frames compound AI systems as architectures that coordinate language models, tools, data, and control components Kandogan et al. (2024). The implementation follows this principle by separating environment observation, world modelling, belief state, memory, goals, planning, tool execution, safety, feedback, and evaluation.

The contribution of the prototype is a course-aligned demonstration of agentic autonomy in a commercial setting. The agent receives external inputs from a website, models the current cart context, diagnoses likely hesitation reasons, chooses safe actions, executes those actions in the site, and records feedback. The commercial objective is conversion recovery: reducing abandoned high-intent carts while avoiding manipulative or unsafe interventions.

2 Problem Verification and Commercial Motivation

E-commerce is a large and measurable market. The U.S. Census Bureau reported that U.S. retail e-commerce sales for the first quarter of 2026 were estimated at \$326.7 billion and accounted for 16.9% of total retail sales U.S. Census Bureau (2026). In such a market, even small improvements in checkout conversion can produce meaningful value.

The selected problem is commercially relevant because cart abandonment occurs after the user has already performed a stronger intent signal than ordinary browsing. Kukar-Kinney and Close identify cognitive and behavioural reasons for cart abandonment, including use of the cart as a research and organization tool rather than immediate checkout Kukar-Kinney and Close (2010). This supports the design choice that a cart-abandonment system should not treat every cart event as a purchase failure. Instead, the system should observe the user's trajectory and infer hesitation causes before intervening.

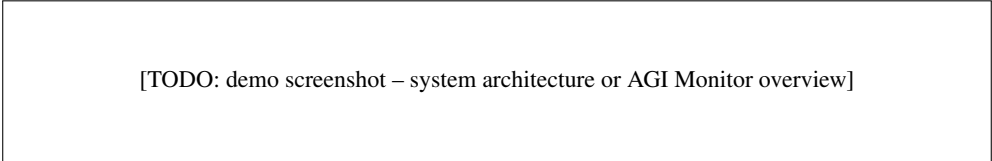
The prototype targets several business outcomes:

- increasing checkout conversion for high-intent carts;
- reducing repeated support workload by proactively clarifying shipping, return, product, or trust concerns;
- improving customer experience through low-annoyance, non-blocking interventions;
- producing structured analytics about which blockers and actions appear effective.

For the commercial stress-test scenario in this report, the assumed average order value is USD \$100, the assumed recovery rate is 1% of high-risk carts, and the maximum incentive cost is 5% of order value. These values are scenario assumptions for demonstrating the cost-benefit method rather than measured production data. The market-size motivation is supported by the U.S. Census Bureau e-commerce sales data cited above U.S. Census Bureau (2026).

3 System Overview

The system contains three cooperating services. The shopping frontend is a React-based simulated e-commerce website with account registration, login, product pages, independent product subpages, cart, checkout, partial checkout, and a safe action runtime. The shopping backend is a Flask service that stores local users and account-isolated carts in SQLite, provides product APIs, validates AGI event forwarding, and injects the authoritative cart snapshot. The AGI backend is a Flask service that stores sessions, events, score history, decisions, feedback, error logs, and evaluation state in SQLite.



[TODO: demo screenshot – system architecture or AGI Monitor overview]

Figure 1: High-level 703DEMOV1 architecture: shopping website events pass through the shopping backend cart gateway before reaching the AGI backend.

The operational flow is:

1. the user interacts with product, cart, checkout, shipping, review, merchant, warranty, or comparison pages;
2. the frontend sends behavioural events to the shopping backend;
3. the shopping backend verifies login state and injects the current server-side cart;
4. the AGI backend updates risk state, world state, belief state, memory, decisions, and evaluation logs;
5. the frontend receives safe web actions, applies an allowlist and cooldown policy, and renders non-blocking UI actions.

This design prevents anonymous browsing or empty carts from creating scoring sessions. It also ensures that analysis remains scoped to the current account and current cart rather than to unrelated product browsing.

4 Agentic Architecture and Methodology

The implementation demonstrates agentic autonomy through modular decision-making. It does not send the entire user trajectory to a model and execute the model’s answer. Instead, it uses a structured pipeline: environment observation, perception encoding, world model update, belief state construction, memory retrieval, goal management, reasoning, planning, tool selection, safety checking, action execution, feedback collection, evaluation logging, and lightweight self-evolution.

4.1 World Model and Belief State

The world model is the central representation layer. It tracks session state, current page, cart contents, checkout progress, recent event trajectory, possible user intention, blockers, predicted next action, environment constraints, available actions, and uncertainty. The belief state converts this representation into uncertain hypotheses over stages, intent, blockers, abandonment risk, and confidence.

This separation matters because the world model stores what the system observes about the environment, whereas the belief state represents what the agent currently believes may be true. For example, repeated shipping-page views after cart exit increase the probability of shipping concern, while cheaper-product browsing related to a cart item increases the probability of price sensitivity or comparison hesitation.

4.2 Memory, Goals, Planning, and Tools

The memory store records raw events, current-session context, decisions, feedback, and reflections. The goal manager prioritizes helping the user make a confident purchase, reducing abandonment, avoiding annoyance, preserving privacy, following constraints, and improving from feedback. The planner generates candidate actions and selects tools such as shipping information, return policy, product reviews, related recommendations, support highlight, demo incentive, trust message, or no action.

The architecture aligns with agent-planning literature that treats planning as decomposition, selection, external-module use, reflection, and memory rather than a single output string Huang et al. (2024). The implementation uses deterministic planning and safety rules for most decisions, which makes the behaviour inspectable during the demo.

4.3 Action Execution

The agent can execute safe actions in the website. Supported frontend actions include non-blocking banners, support-button highlight, related-product recommendation strip, and a small demo cart incentive. These actions are intentionally constrained: they do not auto-purchase, do not alter real catalogue prices, do not collect payment data, and do not send external messages.

5 Risk Scoring, LLM Cost Control, and Safety

Risk scoring is cart-scoped and lifecycle-aware. Browsing without a cart does not create a scoring session. Entering the cart initializes observation but does not reduce the score. Risk analysis starts when the user leaves the cart while products remain inside. If the cart is cleared, scoring pauses and the score resets to 100. If checkout is completed, the current scoring scope ends; partial checkout resets the remaining cart scope.

The risk engine considers multiple behaviour categories:

- product uncertainty: repeated product, information, review, or warranty views;

- shipping concern: shipping-page views, visible shipping fee, or return from checkout to shipping details;
- trust concern: merchant and warranty checks;
- price sensitivity: coupon failure, cheaper alternatives, low-price sorting, and comparison behaviour;
- checkout friction: checkout exit, long checkout dwell, and cart-checkout loops;
- cart weakening: item removal, cart exit, or session exit from cart/checkout.

Gemini is used only for bounded low-score risk multiplier calibration. Google documents that Gemini can generate structured output that adheres to a JSON Schema, which supports predictable and type-safe extraction of structured values Google AI for Developers (2026a). The implementation uses that capability narrowly: Gemini is asked only for a fixed JSON multiplier, with output capped at 10 tokens. If Gemini is unavailable, slow, invalid, or missing an API key, deterministic fallback scoring applies immediately. If a valid Gemini result returns late, the backend may apply a cart-scoped score correction, but the LLM is never allowed to choose website actions.

Safety is guided by AI risk-management principles. NIST describes the AI Risk Management Framework as a voluntary framework for incorporating trustworthiness considerations into AI system design, development, use, and evaluation NIST (2026). OWASP’s 2025 Top 10 for LLM and GenAI applications highlights risks such as prompt injection, sensitive information disclosure, excessive agency, and unbounded consumption OWASP (2025). The prototype addresses these risks through redaction, server-side key storage, restricted LLM scope, frontend action allowlisting, discount caps, no external messaging, no payment-data collection, and cooldown-based popup throttling.

The frontend design also avoids manipulative dark-pattern behaviour. Prior research on dark patterns in online shopping reports negative effects on perceived annoyance and brand trust Voigt et al. (2021). Therefore, visible actions are non-blocking, threshold-gated, deduplicated, and rate-limited.

6 Evaluation Plan and Demo Evidence

The AGI Monitor records the evidence needed to evaluate the system: sessions, risk scores, event history, actions, action feedback, evaluation metrics, error logs, and LLM score-request status. The evaluation is designed to test whether the system correctly reacts to behavioural signals while preserving safety and user experience.

Table 1: Evaluation dimensions for the 703DEMOV1 prototype.

Dimension	Metric or Evidence	Current Evidence
Risk detection	Score changes after cart exit and blocker signals	Score changed from 100 to 37 in the current local log
Action quality	Action shown, clicked, dismissed, or blocked	10 actions and 100 feedback records logged
Safety	Invalid discounts and forbidden actions blocked	0 backend error-log records in current local log
Cost control	LLM calls only in low-score path	16 LLM score requests in current local log
Robustness	Empty cart, partial checkout, multi-device cases	[TODO: add manual edge-case test results]

[TODO: demo screenshot – AGI Monitor session with score history and action feedback]

Figure 2: AGI Monitor evidence for cart-scoped risk scoring and action feedback.

The current local AGI database contains one non-legacy scoring session, 97 user events, 97 score-history rows, 10 saved actions, 100 action-feedback records, 16 LLM score-request records, and 0 backend error-log records. The observed event distribution includes page views, page exits, product

[TODO: demo screenshot – frontend safe action such as support highlight, banner, recommendation strip, or demo incentive]

Figure 3: Frontend execution of a safe non-blocking AGI action.

views, dwell updates, similar-product views, cheaper-alternative views, cart views, and add-to-cart events. The observed action types include product-comparison help, free-shipping offers, discount or cheaper-product recommendations, urgent retention messaging, and cart reminders. These values are local demo-log evidence rather than statistically representative production measurements.

Agent-safety evaluation is important because recent work argues that real-world agents require rigorous evaluation across tool interactions, multi-turn tasks, benign intents, and adversarial intents Vijayvargiya et al. (2025). Although this prototype uses a simulated shopping environment, the evaluation design includes failure logging, frontend safety blocking, backend fallback, and cart-scope isolation to support reliable demonstration.

7 Commercial Stress Test

The commercial stress test compares the cost of running the agent with the expected value of recovered carts. The system is designed to minimize execution cost by using deterministic state and planning logic for most decisions and invoking Gemini only when the risk score is already low. The bounded LLM call returns only a fixed JSON risk multiplier rather than a full reasoning trace. Google lists Gemini 3.5 Flash Standard paid-tier prices as USD \$1.50 per one million input tokens and USD \$9.00 per one million output tokens Google AI for Developers (2026b).

The final report should include the following calculation:

$$\begin{aligned} \text{Expected Net Value} = & \text{Recovered Orders} \times \text{Average Order Value} \\ & - (\text{LLM Cost} + \text{Compute Cost} + \text{Incentive Cost}). \end{aligned}$$

For the current demo database and a clearly labelled scenario assumption, the values are:

- average events per cart-risk session: 97 events in the current local demo log;
- LLM calls per risky session: 16 LLM score requests in the current local demo log;
- approximate LLM cost per session: assuming 500 input tokens and 10 output tokens per LLM scoring request, 16 requests cost approximately USD \$0.0134;
- average order value assumption: USD \$100 per recovered order, used as a scenario value rather than measured production data;
- assumed recovery rate: 1% of high-risk carts, used as a scenario assumption requiring business validation;
- incentive cost: capped at 5% demo incentive when used.

Under this scenario, one recovered USD \$100 order yields USD \$100 gross recovered revenue before incentive and operating costs. A 5% incentive would cost at most USD \$5. The estimated Gemini cost of USD \$0.0134 per risky session is small compared with the assumed order value and incentive cap. Therefore, the stress-test hypothesis is that deterministic scoring and rare low-output LLM calls make the marginal compute cost small compared with the possible revenue value of a recovered high-intent cart. The 1% recovery-rate assumption remains a business assumption and should be validated through controlled testing before any production claim.

8 Limitations and Edge Cases

The implementation is a local course demo rather than a production commerce platform. It uses simulated products, local SQLite databases, local account handling, and mock action execution. Real

deployment would require stronger authentication, privacy review, production observability, data retention policy, fraud controls, secure API gateway integration, and live A/B testing.

Important handled edge cases include anonymous users, empty carts, cleared carts, partial checkout, full checkout, account-isolated carts, multi-device identifiers, AGI backend unavailability, Gemini timeout, invalid Gemini JSON, repeated popups, duplicate action content, and forbidden actions. Remaining limitations include lack of large-scale empirical validation, limited market evidence, and reliance on simulated user behaviour. [TODO: add real edge-case screenshots or logs]

9 Conclusion

703DEMOV1 demonstrates a runnable autonomous agent for cart-abandonment recovery. The system receives external behavioural input, models the shopping environment, diagnoses likely hesitation causes, selects safe interventions, executes actions in the website, collects feedback, and evaluates performance. The design satisfies the course requirement that the system move beyond a simple chat interface: the LLM is only a bounded scoring helper, while state, memory, planning, tool use, safety, feedback, and evaluation are maintained by the system itself. The commercial value lies in recovering high-intent carts and reducing support friction, while the safety and cost controls make the design suitable for a trustworthy demo.

References

References

- M. Kukar-Kinney and A. G. Close. The determinants of consumers’ online shopping cart abandonment. *Journal of the Academy of Marketing Science*, 38:240–250, 2010. URL: <https://link.springer.com/article/10.1007/s11747-009-0141-5>. DOI: <https://doi.org/10.1007/s11747-009-0141-5>.
- U.S. Census Bureau. Quarterly Retail E-Commerce Sales Report, 1st Quarter 2026. 2026. URL: <https://www.census.gov/retail/ecommerce.html>.
- M. Zaharia, O. Khattab, L. Chen, J. Q. Davis, H. Miller, C. Potts, J. Zou, M. Carbin, J. Frankle, N. Rao, and A. Ghodsi. The Shift from Models to Compound AI Systems. Berkeley Artificial Intelligence Research Blog, 2024. URL: <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>.
- E. Kandogan, S. Rahman, N. Bhutani, D. Zhang, R. L. Chen, K. Mitra, S. Gurajada, P. Pezeshkpour, H. Iso, Y. Feng, H. Kim, C. Shen, J. Wang, and E. Hruschka. A Blueprint Architecture of Compound AI Systems for Enterprise. arXiv:2406.00584, 2024. URL: <https://arxiv.org/abs/2406.00584>.
- X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen. Understanding the planning of LLM agents: A survey. arXiv:2402.02716, 2024. URL: <https://arxiv.org/abs/2402.02716>. DOI: <https://doi.org/10.48550/arXiv.2402.02716>.
- National Institute of Standards and Technology. AI Risk Management Framework. Accessed 2026. URL: <https://www.nist.gov/itl/ai-risk-management-framework>.
- OWASP GenAI Security Project. 2025 Top 10 Risk & Mitigations for LLMs and Gen AI Apps. 2025. URL: <https://genai.owasp.org/llm-top-10/>.
- Google AI for Developers. Structured outputs, Gemini API. Accessed 2026. URL: <https://ai.google.dev/gemini-api/docs/structured-output>.
- Google AI for Developers. Gemini Developer API pricing. Accessed 2026. URL: <https://ai.google.dev/gemini-api/docs/pricing>.
- C. Voigt, S. Schlögl, and A. Groth. Dark Patterns in Online Shopping: Of Sneaky Tricks, Perceived Annoyance and Respective Brand Trust. arXiv:2107.07893, 2021. URL: <https://arxiv.org/abs/2107.07893>.

S. Vijayvargiya, A. B. Soni, X. Zhou, Z. Z. Wang, N. Dziri, G. Neubig, and M. Sap. OpenAgentSafety: A Comprehensive Framework for Evaluating Real-World AI Agent Safety. arXiv:2507.06134, 2025. URL: <https://arxiv.org/abs/2507.06134>.

A Reproducibility Notes

The local demo can be run with three services:

1. AGI backend: `cd agi then python app.py`.
2. Shopping backend: `cd online_shopping_web/backend then python app.py`.
3. Frontend: `cd online_shopping_web/frontend then npm.cmd run dev`.

[TODO: add repository link or submission note]

B Demo Video Link

[TODO: add demo video link]

C Manual Test Evidence

bsite after registration/login
anges from 100 after cart exit
view behaviour changes risk
enshot – safe frontend action
back, evaluation, or error log
rrection status, if applicable

D NeurIPS Checklist Placeholder

[TODO: complete the NeurIPS 2025 checklist from the course template.]